QuickCart Black-Box API Testing Report

Test Design

Test case design (inputs, expected outputs, justification)

Below, each test case lists **input**, **expected output**, and **why it matters**.

A) Global header rules

BB-HDR-01 Missing `X-Roll-Number`

- Input: `GET /api/v1/admin/users` without `X-Roll-Number`
- Expected: `401`
- Justification: validates the global authentication/guard rule.

BB-HDR-02 Non-integer `X-Roll-Number`

- Input: `GET /api/v1/admin/users` with `X-Roll-Number: abc`
- Expected: `400`
- Justification: verifies server correctly rejects wrong types.

BB-HDR-03 Missing `X-User-ID` on user endpoint

- Input: `GET /api/v1/profile` without `X-User-ID`
- Expected: `400`
- Justification: ensures user-scoped endpoints enforce user identity.

BB-HDR-04 Invalid `X-User-ID`

- Input: `GET /api/v1/profile` with `X-User-ID: -1`
- Expected: `400`
- Justification: validates positive-integer requirement.

B) Profile

BB-PRO-01 Get profile

- Input: `GET /api/v1/profile` with valid headers
- Expected: `200` and JSON object
- Justification: baseline contract check for user profile retrieval.

BB-PRO-02 Update profile (invalid name bounds)

- Input: `PUT /api/v1/profile` with `name` length `< 2` or `> 50`
- Expected: `400`
- Justification: boundary-value validation for name.

BB-PRO-03 Update profile (invalid phone)

- Input: `PUT /api/v1/profile` with phone not exactly 10 digits
- Expected: `400`
- Justification: wrong length / wrong data constraints.

BB-PRO-04 Update profile (valid boundary)

- Input: `PUT /api/v1/profile` with name length `2` and phone `10 digits`
- Expected: `200/201` and JSON
- Justification: confirms the lower valid boundary is accepted.

BB-PRO-05 Update profile (missing fields)

- Input: `PUT /api/v1/profile` with `{}`, or missing `name`, or missing `phone`
- Expected: `400`
- Justification: required-field validation.

BB-PRO-06 Update profile (wrong data types)

- Input: `PUT /api/v1/profile` with non-string name / non-string phone

- Expected: `400`
- Justification: wrong type validation.

C) Addresses

BB-ADDR-01 Get addresses

- Input: `GET /api/v1/addresses`
- Expected: `200` and JSON list/object
- Justification: baseline retrieval.

BB-ADDR-02 Add address invalid label/pincode/street/city

- Input: `POST /api/v1/addresses` with invalid label, street < 5, city < 2, pincode != 6 digits
- Expected: `400`
- Justification: invalid inputs + boundary checks.

BB-ADDR-03 Add address success returns full object

- Input: `POST /api/v1/addresses` with valid fields
- Expected: `200/201` and JSON containing `address_id`, `label`, `street`, `city`, `pincode`, `is_default`
- Justification: validates create contract and required response structure.

BB-ADDR-04 Default uniqueness

- Input: create 2 addresses with `is_default=true`, then `GET /api/v1/addresses`
- Expected: at most 1 address marked default
- Justification: validates business rule "only one default address".

BB-ADDR-05 Update address only street/is_default

- Input: `PUT /api/v1/addresses/{id}` attempting to change city
- Expected: request rejected (400) or accepted but city unchanged; street updates
- Justification: validates restricted update rules.

BB-ADDR-06 Delete non-existent address

- Input: `DELETE /api/v1/addresses/99999999`
- Expected: `404`
- Justification: validates correct error code for missing resource.

BB-ADDR-07 Add address (missing fields / wrong types / boundary valid)

- Input: omit required fields; send wrong types (numbers instead of strings); use boundary-valid lengths (street 5/100, city 2/50)
- Expected: missing/wrong types => `400`; boundary-valid => `200/201`
- Justification: covers missing-field, wrong-type, and boundary-value criteria explicitly.

D) Products

BB-PROD-01 Product list excludes inactive

- Input: `GET /api/v1/products`
- Expected: response contains only active products
- Justification: confirms filtering rule "inactive never shown".

BB-PROD-02 Product lookup not found

- Input: `GET /api/v1/products/99999999`
- Expected: `404`
- Justification: correct missing-resource behavior.

BB-PROD-03 Filter/search/sort (best-effort due to ambiguous query params)

- Input: `GET /api/v1/products?category=...`, `?search=...` or `?q=...`, `?sort=price&order=asc`
- Expected: `200` and behavior matches filter/search/sort; if API rejects unknown params, `400`
- Justification: the doc states these capabilities exist; this validates the decision paths when supported.

E) Cart

BB-CART-01 Add with quantity <= 0

- Input: `POST /api/v1/cart/add` with `quantity=0`
- Expected: `400`
- Justification: invalid quantity boundary.

BB-CART-02 Add unknown product

- Input: `POST /api/v1/cart/add` with non-existent product
- Expected: `404`
- Justification: validates product existence checks.

BB-CART-03 Add same product twice increments quantity

- Input: add product quantity 1, add again quantity 2
- Expected: final cart line quantity = 3
- Justification: validates rule "quantities add, not replace".

BB-CART-04 Cart totals and subtotals

- Input: `GET /api/v1/cart` after adds
- Expected: `subtotal == qty * unit_price` and `total == sum(subtotals)`
- Justification: validates correct computed totals and "last item counted" rule.

BB-CART-05 Update quantity invalid and valid

- Input: `POST /api/v1/cart/update` with quantity 0 (invalid) then quantity 1 (valid)
- Expected: invalid => 400; valid => 200/201
- Justification: quantity rules apply to update as well.

BB-CART-06 Remove missing item

- Input: `POST /api/v1/cart/remove` twice
- Expected: second remove => 404
- Justification: validates correct error code when item not present.

BB-CART-07 Add quantity > stock

- Input: add with `quantity=stock+1`
- Expected: `400`
- Justification: validates stock guard.

BB-CART-08 Add item (missing fields / wrong types)

- Input: `{}`, missing `product_id`, missing `quantity`, string types instead of integers
- Expected: `400`
- Justification: required-field + type validation.

F) Checkout + Orders + Invoice

BB-CHK-01 Checkout empty cart

- Input: `POST /api/v1/checkout` with empty cart
- Expected: `400`
- Justification: prevents invalid order creation.

BB-CHK-02 Checkout invalid payment method

- Input: `payment_method=BITCOIN`
- Expected: `400`
- Justification: validates allowed enum: COD/WALLET/CARD only.

BB-CHK-03 COD not allowed above 5000

- Input: cart total > 5000 + `payment_method=COD`
- Expected: `400`
- Justification: boundary rule tied to business constraints.

BB-CHK-04 CARD checkout success

- Input: `payment_method=CARD` with non-empty cart

- Expected: `200/201` and JSON
- Justification: validates a known-success checkout path.

BB-ORD-01 Orders list

- Input: `GET /api/v1/orders`
- Expected: `200` and list
- Justification: baseline order retrieval.

BB-INV-01 Invoice contains subtotal/gst/total

- Input: `GET /api/v1/orders/{id}/invoice`
- Expected: fields present
- Justification: validates invoice structure and pricing transparency.

BB-CAN-01 Cancel non-existent order

- Input: `POST /api/v1/orders/99999999/cancel`
- Expected: `404`
- Justification: missing resource error code.

BB-CAN-02 Cancel delivered order rejected

- Input: attempt to cancel delivered order
- Expected: `400`
- Justification: validates the “delivered cannot be cancelled” rule.

G) Wallet

BB-WAL-01 Wallet add amount boundary

- Input: `POST /api/v1/wallet/add` with 0 and 100001
- Expected: `400`
- Justification: boundary values.

BB-WAL-02 Wallet pay invalid and insufficient

- Input: `POST /api/v1/wallet/pay` with 0 and a huge value
- Expected: `400`
- Justification: invalid amount + insufficient funds rule.

BB-WAL-03 Wallet pay exact deduction (if balance exposed)

- Input: add 1 then pay 1
- Expected: balance decreases by exactly 1
- Justification: verifies no extra deductions.

H) Loyalty

BB-LOY-01 Redeem invalid and too many

- Input: redeem 0, redeem huge
- Expected: `400`
- Justification: invalid amount + insufficient points.

I) Reviews

BB-REV-01 Rating bounds

- Input: rating 0 and 6
- Expected: `400`
- Justification: boundary validation.

BB-REV-02 Comment length bounds

- Input: empty comment and 201 chars
- Expected: `400`
- Justification: boundary validation.

BB-REV-03 Average rating decimal (best effort)

- Input: add two ratings 4 and 5, then GET reviews
- Expected: average rating should be computed as decimal (not floored)
- Justification: validates numeric correctness.

J) Support tickets

BB-TKT-01 Create ticket subject/message bounds

- Input: subject length < 5 and message length > 500
- Expected: `400`
- Justification: boundary validation.

BB-TKT-02 Ticket status monotonic progression

- Input: OPEN -> IN_PROGRESS -> CLOSED, then attempt CLOSED -> OPEN
- Expected: last transition rejected with `400`
- Justification: validates state machine constraints.

K) Coupons

BB-CPN-01 Apply coupon with missing code

- Input: `POST /api/v1/coupon/apply` with {}
- Expected: `400`
- Justification: missing field validation.

BB-CPN-02 Apply coupon (database-dependent)

- Input: `POST /api/v1/coupon/apply` using a code discovered via `GET /api/v1/admin/coupons`
- Expected: `200/201` on success OR `400` if coupon is expired / min-cart not met / cap rules fail
- Justification: exercises coupon validation rules stated in the doc.

BB-CPN-03 Remove coupon

- Input: `POST /api/v1/coupon/remove`
- Expected: `200/201` (or a documented `400` if API requires a coupon to be applied first)
- Justification: validates coupon removal behavior and response structure.

Missing-fields / wrong-types coverage

Automated tests also include a dedicated negative-payload suite (`blackbox/tests/test_quickcart_negative_payloads.py`) that systematically checks:

- Missing required JSON fields (`{}` and partial payloads)
- Wrong data types (strings vs integers, integers vs strings)
- Boundary-valid payloads where the doc provides ranges

Automated verification checklist

All automated tests verify:

- Correct HTTP status codes
- JSON response structure (at least: parseable JSON, expected container types)
- Returned data correctness where the specification provides a determinable rule (e.g., cart totals)

Bug Reports

This file contains **distinct, non-overlapping** defects derived from the automated black-box test cases.

Bug 1: Address update response shows old data

- **Endpoint tested**: `PUT /api/v1/addresses/{address_id}`
- **Category**: Data correctness | Business rule

Request

- Method: `PUT`
- URL: `http://localhost:8080/api/v1/addresses/1024`
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1`

- `Content-Type: application/json`

- Body (JSON):

```
{
  "city": "Changed",
  "street": "33333 Gamma Street UPDATED",
  "is_default": true
}
```

Expected result (per API doc)

- Status: `200`
- Behavior:

- Only `street` and `is_default` are updatable.

- Response must show the updated data.

Actual result observed

- Status: `200`
- Body (example):

```
{
  "address": {
    "address_id": 1024,
    "label": "OTHER",
    "street": "33333 Gamma Street",
    "city": "Gamma",
    "pincode": "333333",
    "is_default": false
  },
  "message": "Address updated successfully"
}
```

Notes

- Why this is a defect: response returns the pre-update street/default state.
- Reproducibility: Always (observed consistently).
- Severity: Medium

Bug 2: Cart add does not validate the `quantity` field (missing/0 accepted)

- **Endpoint tested**: `POST /api/v1/cart/add`
- **Category**: Validation | Business rule

Requests

Case A: quantity is zero

```
{
  "product_id": 1,
  "quantity": 0
}
```

Case B: quantity is missing

```
{
  "product_id": 1
}
```

Expected result (per API doc)

- Status: `400`
- Behavior: quantity must be present and at least 1.

Actual result observed

- Status: `200`
- Body (example): `{"message":"Item added to cart"}`

Notes

- Why this is a defect: violates cart quantity validation rules.
- Reproducibility: Always.
- Severity: High

Bug 3: Cart add treats missing `product_id` as “Product not found” (404)

- **Endpoint tested**: `POST /api/v1/cart/add`
- **Category**: Validation | Status code

Request

- Method: `POST`
- URL: `http://localhost:8080/api/v1/cart/add`
- Headers:

- `X-Roll-Number: 1`
- `X-User-ID: 1`
- `Content-Type: application/json`

- Body (JSON) example A:

```
{}
```

- Body (JSON) example B:

```
{
  "quantity": 1
}
```

Expected result (per API doc)

- Status: `400`
- Behavior: missing required fields should be treated as an invalid request (400), not as a missing product.

Actual result observed

- Status: `404`
- Body:

```
{
  "error": "Product not found"
}
```

Notes

- Why this is a defect: For missing `product_id`, the request is malformed; returning "not found" is misleading and contradicts expected validation behavior.
- Reproducibility: Always
- Severity: Medium

Bug 4: Wallet pay deducts more than requested amount

- **Endpoint tested**: `POST /api/v1/wallet/pay`
- **Category**: Data correctness | Business rule

Request

- Method: `POST`
- URL: `http://localhost:8080/api/v1/wallet/pay`
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1`

- `Content-Type: application/json`

- Body (JSON):

```
{
  "amount": 1
}
```

Expected result (per API doc)

- Status: `200`
- Behavior: "the exact amount requested is deducted from the balance. No extra amount is taken."

Actual result observed

- Example observed during automated run (values vary depending on prior state):

- Balance before paying 1: `100086.59`

- Balance after paying 1: `100084.99`

- Effective deduction: `1.60` for a requested `1.00`

Notes

- Why this is a defect: Balance decreased by 1.6 (83.59 → 81.99) for a payment of 1.
- Reproducibility: Always
- Severity: High

Bug 5: Loyalty redeem rejects a valid redeem amount of 1

- **Endpoint tested**: `POST /api/v1/loyalty/redeem`
- **Category**: Validation | Business rule

Request

- Method: `POST`
- URL: `http://localhost:8080/api/v1/loyalty/redeem`
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1`

- `Content-Type: application/json`

- Body (JSON):

```
{
  "amount": 1
}
```

Expected result (per API doc)

- Status: `200` or `201`
- Behavior: amount to redeem must be at least 1, and user must have enough points.

Actual result observed

- Current points:

```
{
  "loyalty_points": 448
}
```

- Redeem response:

```
{
  "error": "Points must be >= 1"
}
```

Notes

- Why this is a defect: User has sufficient points and amount=1 satisfies “at least 1”, yet server rejects with an error message that contradicts the request.
- Reproducibility: Always
- Severity: Medium

Bug 6: Reviews endpoint accepts invalid rating 0

- **Endpoint tested**: `POST /api/v1/products/{product_id}/reviews`
- **Category**: Validation

Request

- Method: `POST`
- URL: `http://localhost:8080/api/v1/products/1/reviews`
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1`

- `Content-Type: application/json`

- Body (JSON):

```
{
  "rating": 0,
  "comment": "ok"
}
```

Expected result (per API doc)

- Status: `400`
- Behavior: rating must be between 1 and 5.

Actual result observed

- Status: `200`
- Body:

```
{
  "message": "Review added successfully",
  "review_id": 1006
}
```

Notes

- Why this is a defect: Violates the rating range validation.
- Reproducibility: Always
- Severity: Medium

Bug 7: Checkout allows checkout with empty cart

- **Endpoint tested**: `POST /api/v1/checkout`
- **Category**: Business rule | Validation

Request

- Method: `POST`
- URL: `http://localhost:8080/api/v1/checkout`
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1`

- `Content-Type: application/json`

- Body (JSON):

```
{
  "payment_method": "COD"
}
```

Expected result (per API doc)

- Status: `400`
- Behavior: "The cart must not be empty. If it is empty, the server returns a 400 error."

Actual result observed

- Status: `200`
- Body:

```
{
  "gst_amount": 0,
  "order_id": 3006,
  "order_status": "PLACED",
  "payment_status": "PENDING",
  "total_amount": 0
}
```

Notes

- Why this is a defect: Creates an order with total 0 from an empty cart.
- Reproducibility: Always
- Severity: High

Bug 8: Address creation rejects valid 6-digit pincodes (pincode validation broken)

- **Endpoint tested**: `POST /api/v1/addresses`
- **Category**: Validation

Requests

- Method: `POST`
- URL: `http://localhost:8080/api/v1/addresses`
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1`

- `Content-Type: application/json`

- Body (JSON) (valid pincode, 6 digits):

```
{
  "label": "HOME",
  "street": "12345 Main Street",
  "city": "City",
  "pincode": "111111",
  "is_default": false
}
```

Expected result (per API doc)

- Status: `200` or `201`
- Behavior: pincode must be exactly 6 digits; a 6-digit value like `111111` should be accepted.

Actual result observed

- Status: `400`
- Body:

```
{"error":"Invalid pincode"}
```

Notes

- Why this is a defect: rejects a valid 6-digit pincode with `Invalid pincode`.

- Confirmation: reproduced both in the pytest run with roll `2024101077` and via a manual POST using the same roll/user headers.
- Severity: High (blocks address workflows entirely)

Bug 9: Profile update accepts phone numbers containing non-digits

- **Endpoint tested**: `PUT /api/v1/profile`
- **Category**: Validation

Request

- Method: `PUT`
- URL: `http://localhost:8080/api/v1/profile`
- Headers:

- `X-Roll-Number: 1`
- `X-User-ID: 1`
- `Content-Type: application/json`

- Body (JSON):

```
{
  "name": "Valid Name",
  "phone": "01234A6789"
}
```

Expected result (per API doc)

- Status: `400`
- Behavior: phone must be exactly 10 digits (digits only).

Actual result observed

- Status: `200`
- Body:

```
{
  "message": "Profile updated successfully"
}
```

Notes

- Why this is a defect: Accepts non-digit characters in phone, violating the documented constraint.
- Reproducibility: Always (observed consistently in parameterized runs).
- Severity: Medium

Bug 10: Cancelling an order does not restore product stock

- **Endpoint tested**: `POST /api/v1/orders/{order_id}/cancel`
- **Category**: Data correctness | Business rule

Request

- Method: `POST`
- URL: `http://localhost:8080/api/v1/orders/{order_id}/cancel`
- Headers:

```
-X-Roll-Number: 1

-X-User-ID: 1
```

Expected result (per API doc)

- Status: `200`
- Behavior: "When an order is cancelled, all the items in that order are added back to the product stock."

Actual result observed

- Status: `200`
- Cancel response body:

```
{
  "message": "Order cancelled successfully",
  "order_status": "CANCELLED"
}
```

- Stock check via `GET /api/v1/admin/products` for `product_id=1`:

- Before checkout+cancel: `stock_quantity = 188`

- After cancel: `stock_quantity = 187`

Notes

- Why this is a defect: Stock decreased (or at least did not increase), contradicting the documented restock rule.
- Reproducibility: Observed consistently.
- Severity: High

Bug 11: Product prices returned to users do not match database prices

- **Endpoints tested**: `GET /api/v1/products`, `GET /api/v1/admin/products`
- **Category**: Data correctness

Request

- Method: `GET`
- URL A: `http://localhost:8080/api/v1/admin/products` (admin)
- URL B: `http://localhost:8080/api/v1/products` (user)
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1` (for user endpoint)

Expected result (per API doc)

- Behavior: "The price shown for every product must be the exact real price stored in the database."

Actual result observed

- Example mismatch:

- `product_id = 8` (`Grapes - Black`)

- `admin/products` price: `95`

- `products` price: `100`

Notes

- Why this is a defect: User-visible price differs from DB price.
- Reproducibility: Observed in automated sample checks.
- Severity: High

Bug 12: Cart item subtotals can overflow / become negative

- **Endpoint tested**: `GET /api/v1/cart`
- **Category**: Data correctness | Business rule

Request

- Method: `GET`
- URL: `http://localhost:8080/api/v1/cart`
- Setup: add items via `POST /api/v1/cart/add`
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1`

Expected result (per API doc)

- Behavior: "Each item in the cart must show the correct subtotal, which is the quantity times the unit price."

Actual result observed

- Example A (unit_price > 127): after adding `product_id=2` with `quantity=1`:

```
{
  "product_id": 2,
  "quantity": 1,
  "unit_price": 130,
  "subtotal": -126
}
```

- Example B (larger quantities): after adding `product_id=3` with `quantity=10` (unit_price 40):

```
{
  "product_id": 3,
  "quantity": 10,
  "unit_price": 40,
  "subtotal": -112
}
```

Notes

- Why this is a defect: Subtotal becomes negative/wrapped instead of $quantity \times unit_price$.
- Reproducibility: Always for these inputs.
- Severity: High

Bug 13: Cart total does not equal sum of item subtotals (last item not counted)

- **Endpoint tested**: `GET /api/v1/cart`
- **Category**: Data correctness | Business rule

Request

- Method: `GET`
- URL: `http://localhost:8080/api/v1/cart`
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1`

Expected result (per API doc)

- Behavior:

- “The cart total must be the sum of all item subtotals.”
- “Every item must be counted, including the last one.”

Actual result observed

- With a single item (`product_id=1`, `quantity=1`, `subtotal=120`):

```
{
  "items": [
    {
      "product_id": 1,
      "quantity": 1,
      "unit_price": 120,
      "subtotal": 120
    }
  ],
  "total": 0
}
```

Notes

- Why this is a defect: Total is not the sum of subtotals; the only/last item is effectively not counted.
- Reproducibility: Observed consistently.
- Severity: High

Bug 14: Coupon apply endpoint does not accept documented request field `code`

- **Endpoint tested**: `POST /api/v1/coupon/apply`
- **Category**: JSON schema | Validation

Request

- Method: `POST`
- URL: `http://localhost:8080/api/v1/coupon/apply`
- Headers:

 - `X-Roll-Number: 1`

 - `X-User-ID: 1`

 - `Content-Type: application/json`

- Body (JSON) (documented):

```
{
  "code": "FIRSTORDER"
}
```

Expected result (per API doc)

- Behavior: coupon code should be read from the `code` field.

Actual result observed

- Using documented field `code`:

 - Status: `400`

 - Body: `{"error":"Invalid coupon"}`

- Using an undocumented field name `coupon_code` instead:

 - Status: `200`

 - Body:

```
{
  "coupon_code": "FIRSTORDER",
  "discount": 36,
  "new_total": 204
}
```

Notes

- Why this is a defect: Request schema deviates from the API documentation.
- Reproducibility: Always.
- Severity: Medium

Bug 15: WALLET checkout succeeds even when wallet balance is insufficient

- **Endpoint tested**: `POST /api/v1/checkout`
- **Category**: Validation | Business rule

Request

- Method: `POST`
- URL: `http://localhost:8080/api/v1/checkout`
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1`

- `Content-Type: application/json`

- Body (JSON):

```
{
  "payment_method": "WALLET"
}
```

Expected result (per API doc)

- Status: `400`
- Behavior: WALLET payment should be rejected when funds are insufficient.

Actual result observed

- Wallet balance (before checkout):

```
{
  "wallet_balance": 78.39000000000007
}
```

- Checkout response:

- Status: `200`

- Body:

```
{
  "gst_amount": 18,
  "order_id": 3018,
  "order_status": "PLACED",
  "payment_status": "PENDING",
  "total_amount": 378
}
```

Notes

- Why this is a defect: Allows placing a WALLET order far above the available balance.
- Reproducibility: Always for this scenario.
- Severity: High

Bug 16: Expired coupons are accepted

- **Endpoint tested**: `POST /api/v1/coupon/apply`
- **Category**: Business rule | Validation

Request

- Method: `POST`
- URL: `http://localhost:8080/api/v1/coupon/apply`
- Headers:

- `X-Roll-Number: 1`
- `X-User-ID: 1`
- `Content-Type: application/json`

- Body (JSON):

```
{
  "coupon_code": "BIGDEAL500"
}
```

Expected result (per API doc)

- Status: `400`
- Behavior: expired coupons must be rejected.

Actual result observed

- Status: `200`
- Body:

```
{
  "coupon_code": "BIGDEAL500",
  "discount": 500,
  "new_total": 4540
}
```

Notes

- Why this is a defect: `BIGDEAL500` has `expiry_date=2026-03-20T00:00:00Z` and the current date is 2026-03-21, yet it is still accepted.
- Reproducibility: Always (observed in strict coupon-rule test).
- Severity: High

Bug 17: Support ticket message is not saved exactly as written (message gets truncated)

- **Endpoints tested**: `POST /api/v1/support/ticket`, `GET /api/v1/admin/tickets`
- **Category**: Data correctness | Business rule

Request

- Method: `POST`
- URL: `http://localhost:8080/api/v1/support/ticket`
- Headers:

- `X-Roll-Number: 1`

- `X-User-ID: 1`

- `Content-Type: application/json`

- Body (JSON) example:

```
{
  "subject": "exact-message-check 2026-03-21T12:34:56.789012+00:00",
  "message": "Line1 2026-03-21T12:34:56.789012+00:00\nLine2
```

```
!@#$%^&*()[]{}\\|;:'\",.<>/?"
}
```

Expected result (per API doc)

- Status: `200` or `201`
- Behavior: “The full message must be saved exactly as written.”

Actual result observed

- Status: `200` (ticket created successfully)
- But inspecting the stored ticket via `GET /api/v1/admin/tickets` shows the message was not preserved:

- Expected stored message suffix: `Line2 !@#$%^&*()[]{}\|;:'",.<>/?`

- Actual stored message suffix observed: `Line2 !@#$%`

Notes

- Why this is a defect: the system truncates/changes the ticket message, violating the “saved exactly” requirement.
- Reproducibility: Observed consistently with messages containing special characters.
- Severity: Medium

Bug 18: Non-existent positive `X-User-ID` is not rejected consistently (treated as a valid principal)

- **Endpoints tested**: multiple user-scoped endpoints (GET + mutations)
- **Category**: Validation | Authorization

Request

- Header used: `X-User-ID: 99999999` (positive integer but not an existing user)

Expected result (per API doc)

- Status: `400`
- Behavior: user-scoped endpoints require `X-User-ID` match an existing user; non-existent IDs are invalid.

Actual result observed

- In a single matrix run with `X-User-ID: 99999999`:

- `GET /api/v1/products` -> `200` (product list returned)

- `GET /api/v1/profile` -> `404` (`{"error":"User not found"}`)

- `GET /api/v1/addresses` -> `200` with `[]`

- `GET /api/v1/cart` -> `200` with an empty cart object

- `GET /api/v1/wallet` -> `200` with `{"wallet_balance": 0}`

- `GET /api/v1/loyalty` -> `200` with `{"loyalty_points": 0}`

- `GET /api/v1/orders` -> `200` with `[]`

- `GET /api/v1/support/tickets` -> `200` with `[]`

- `GET /api/v1/products/1/reviews` -> `200` with review data

- Mutations with non-existent `X-User-ID` also succeed in separate tests (examples observed):

- `POST /api/v1/cart/add` -> `200`

- `POST /api/v1/addresses` -> `200`

- `POST /api/v1/support/ticket` -> `200`

- `POST /api/v1/wallet/add` -> `200` (even when balance stays at 0)

- `POST /api/v1/checkout` -> `200` (order created)
- `POST /api/v1/products/{id}/reviews` -> `200` (review created)

Notes

- Why this is a defect: violates the documented requirement that `X-User-ID` must match an existing user for all user-scoped endpoints.
- Reproducibility: Always.
- Severity: Critical

Bug 19: Server becomes unreachable / aborts connections during normal API usage

- **Endpoints observed**: `GET /api/v1/admin/orders`, `DELETE /api/v1/cart/clear` (and then subsequent requests fail)
- **Category**: Reliability | Availability

Expected result

- API should return a normal HTTP response (200/4xx) and remain available for subsequent requests.

Actual result observed

- During isolated test runs, requests abort with:
  - `RemoteDisconnected('Remote end closed connection without response')`
  - followed by `ConnectionRefusedError` / `curl` returning `000` for health probes.
- One reproducible instance occurred while calling `GET /api/v1/admin/orders` inside `test_cancel_delivered_order_rejected_if_any_exist`, after which the server stopped responding.

Notes

- Why this is a defect: the server crashes or becomes unavailable under routine API calls, causing cascaded test failures and breaking client behavior.
- Severity: Critical

Bug 20: Invoice GST math is incorrect (GST not equal to 5% of subtotal)

- **Endpoint tested**: `GET /api/v1/orders/{order_id}/invoice` (after `POST /api/v1/checkout`)
- **Category**: Data correctness | Business rule

Expected result (per API doc)

- GST must be 5% of the subtotal and added only once.
- Invoice must be internally consistent: $total \approx subtotal + gst$.

Actual result observed

- In an automated run (controlled cart with `subtotal ≈ 240.00`):

- Returned `gst_amount = 24.6`

- Expected GST: $0.05 \times 240.00 = 12.00$

- Observed ratio: $24.6 / 240 \approx 10.25\%$ (suggesting GST applied incorrectly)

Notes

- Why this is a defect: violates the documented GST rule and leads to incorrect invoice totals.

- Evidence source: failing `test_invoice_math_and_gst_added_once` in `test_quickcart_new_distinct_defects.py` from the roll `2024101077` run log.